

requirement of network stack administration is to monitor the packet flow through the system. Linux provides a file named `/proc/net/dev`, which shows the packet flow through every interface in the system. The following table shows an example from `/proc/net/dev` file. Although the file has many more fields, some have been stripped down for a better understanding.

```
foo@localhost:~$ cat /proc/net/dev
```

	Receive				Transmit			
face	bytes	packets	errs	drop	bytes	packets	errs	drop
lo:	80036	1597	0	0	80036	1597	0	0
eth0:	0	0	0	0	0	0	0	0
eth1:	26922	154	0	0	19871	194	0	0

As the example shows, the system has received 26922 bytes and 154 packets over the eth1 interface while the eth0 interface has not received any packet. Similarly, the system has transmitted 19871 bytes and 194 packets over the eth1 interface. Clearly, no error packets were received, and none were dropped either.

Therefore this file can be used for a quick view of packet flow through the system.

As the received/transmitted packets are processed by IP and TCP/UDP protocols, a deeper look of the packet flow can be viewed in `/proc/net/snmp` file. A stripped down view of the file is shown below:

```
Ip: InReceives InDiscards OutRequests OutDiscards
Ip: 2725 0 2519 0
Icmp: InMsgs InErrors OutMsgs OutErrors
Icmp: 72 0 73 0
Udp: InDatagrams NoPorts InErrors OutDatagrams
Udp: 116 0 0 116
```

The above example shows the packet distribution at the upper layers. This file shows many more packet counts for different cases like packet header errors, address errors, port errors, etc.

Therefore this file is useful for a quick view of connection patterns through the system.

Packet capturing: Linux provides a very powerful and versatile socket interface that can be used to pick packets directly from the Ethernet or IP layer. These sockets are called PF_PACKET sockets. The following small snippet of C code shows how to use PF_PACKET sockets to capture packets arriving at the system.

```
#define IF_NAME "eth0"
struct ifreq ifr = (0);

fd = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));

/* Set promiscuous mode to capture even those packets which are not destined
to our host */

strncpy(ifr.ifr_name, IF_NAME, IFNAMSIZ);
ioctl(fd, SIOCGIFFLAGS, &ifr);
ifr.ifr_flags |= IFF_PROMISC;
ioctl(fd, SIOCSIFFLAGS, &ifr);

/* bind to "eth0" if only those packet arriving on "eth0" are required */
strncpy(ifr.ifr_name, IF_NAME, IFNAMSIZ);
setsockopt(fd, SOL_SOCKET, SO_BINDTODEVICE, (char*)&ifr, sizeof(ifr));

recv(fd, buf, sizeof(buf), 0);
```

The above code snippet, with ignored error checks, shows how to capture a packet arriving at the interface eth0. The packet contents can be used to have a deeper look into the established or attempted connections/flows.

In the above example, socket_type is used as SOCK_RAW to receive the entire packet including the link layer header. The SOCK_DGRAM socket_type can be used to capture packets with the link layer header removed.

When the protocol field is set to ETH_P_ALL, all protocol packets are received.

Many packet-capturing tools are based on PF_PACKET sockets. Those packet-capturing tools provide the additional functions of packet filtering, organised display of packet contents, etc, which make administration simpler.

Management/Control

This section describes some methods for the management and control of the TCP/IP networking stack.

Packet forwarding: A networking system running Linux can be configured to run as a host or as the router. In the latter case, the system can forward packets destined for other machines, to the appropriate next hop. Packet forwarding on Linux can be controlled, per interface, by modifying `/proc/sys/net/ipv4/conf/<device_name>/forwarding`. Writing 0 in this file disables packet forwarding while 1 enables packet forwarding.

Address resolution: The ARP protocol is used for IP address resolution. Complex networking systems have many interfaces and many IP addresses. It is possible to control which source address is put in to the ARP protocol headers using the file located at `/proc/sys/net/ipv4/conf/<device_name>/arp_announce`. It takes the following values:

- 0 – (default) any local address
- 1 – uses the address from the same subnet as the target address
- 2 – prefers the primary address

Therefore, this file provides a way to control address resolution.

Resolution timeout: The Linux kernel tries to resolve an IP address through ARP if the address is not already resolved. This process queues the outgoing application packets and starts the address resolution procedure. However, if the target system is not reachable, the kernel must give up address resolution after some tries and intimate the application about the unreachable destination. The maximum number of attempts to resolve the IP address through multicast/